

Comp-2540: Dynamic Programming (I)

November 23, 2025

Jianguo Lu, University of Windsor

"Those who cannot remember the past are condemned to repeat it."

Algorithm design strategies

- Generate and test
- Greedy method
- Divide and conquer
- Dynamic programming

Dynamic Programming

- Used on two types of problems:
 - Optimization problems: select a feasible solution, so that the value of the required function is minimized or maximized.
 - Combinatorial problems: figure out the number of ways to do something
- .
- Divide-and-conquer: It divides a problem into sub-problems. (so it is an extension of Divide and Conquer)
- Overlapping sub-solutions: The sub-solutions overlap. These sub-solutions are computed over and over again when computing the global optimal solution in a brute-force algorithm.
- Memorization: Use memorization to save repeated solution for the same subproblem.
- Optimal substructure: The optimal solution to the problem contains within its subsolutions.

Outline

1 Fibonacci Number: Motivating Example for DP

• Variations of Fibonacci Number: Climb stairs

- Distinct ways to climb stairs
- Minimum cost stair climbing

2 Extension of stair climbing: Frog jump

- Frog Jump with missing stone
- Frog Jump with energy supply
- Number of sums

3 2D paths

4 Knapsack problem

- Recursive solution
- Tabulated solution

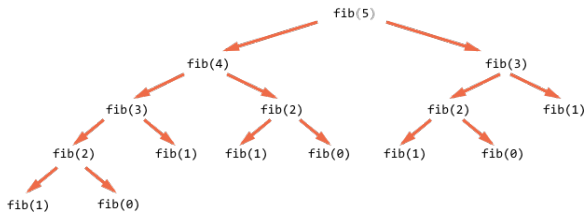
Dynamic Programming: Fibonacci example

$$fib(n) = \begin{cases} 1, & \text{if } n=0 \\ 1, & \text{if } n=1 \\ fib(n-1) + fib(n-2), & n > 1 \end{cases}$$

```
1 long fibonacciBad(int n) {  
2     if (n <= 1) return n;  
3     else return fibonacciBad(n-2) + fibonacciBad(n-1);  
4 }
```

- Time complexity of the program?

Repeated calculations



- Do not need to recompute previous subproblems
- Use a table to save what are already computed

DP offers two methods to solve a problem

Top-down with Memoization:

- solve the bigger problem by recursively finding the solution to smaller sub-problems.
- Whenever we solve a sub-problem, we cache its result so that we don't end up solving it repeatedly if it's called multiple times.
- Instead, we can just return the saved result. This technique of storing the results of already solved subproblems is called Memoization.

```
1 int memoize[] = new int[n+1];  
2  
3 int fibonacci(int[] memoize, int n) {  
4     if(n < 2) return n;  
5     if(memoize[n] != 0) return memoize[n];  
6     memoize[n] = fibonacci(n-1) + fibonacci(n-2);  
7     return memoize[n];  
8 }  
9  
10
```

Do not need to save the entire array

- When we search for calculated fib numbers, we only need `fib(n-1)` and `fib(n-2)`
- Hence we only need to save two previous calculations
- How to save those two values?

```
2  long [] fibonacciSaveTwo(int n) {  
3      if (n <= 1) {  
4          long [] answer = {n, 0};  
5          return answer;  
6      } else {  
7          long [] temp = fibonacciSaveTwo(n - 1);  
8          long [] answer = {temp[0] + temp[1], temp[0]};  
9          return answer;  
10     }  
11 }
```


Bottom-up with Tabulation

- Tabulation is the opposite of the top-down approach and avoids recursion.
- Solve the problem by solving the related sub-problems first.
- Typically done by filling up an n-dimensional table.
- Based on the results in the table, the solution to the top/original problem is then computed.

```
2 public int CalculateFibonacci(int n) {  
3     if (n==0) return 0;  
4     int dp[] = new int[n+1];  
5     dp[0] = 0;  
6     dp[1] = 1;  
7  
8     for(int i=2; i<=n; i++)    dp[i] = dp[i-1] + dp[i-2];  
9     return dp[n];  
10 }
```

Outline

1 Fibonacci Number: Motivating Example for DP

- Variations of Fibonacci Number: Climb stairs
 - Distinct ways to climb stairs
 - Minimum cost stair climbing

2 Extension of stair climbing: Frog jump

- Frog Jump with missing stone
- Frog Jump with energy supply
- Number of sums

3 2D paths

4 Knapsack problem

- Recursive solution
- Tabulated solution

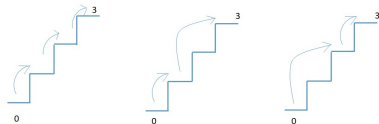
Another simple example and its variations

- Distinct ways to climb stairs (Combinatorial problem)
- Minimum cost of stair climbing (Optimization problem)
- Frog Jump

Distinct ways to climb stairs

Problem

You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?



Ways to climb 3-step stair:

- $1+1+1$
- $1+2$
- $2+1$

Recurrence relation

To reach step n , your last move must be either

- 1-step jump from step $n-1$
- 2-steps jump from step $n-2$

Hence,

$$sol(n) = \begin{cases} 1, & \text{if } n=1 \\ 2, & \text{if } n=2 \\ sol(n-1) + sol(n-2), & n > 2 \end{cases} \quad (1)$$

It's actually the same as the fibonacci problem!

Minimum cost stair climbing

Problem

- Given an integer array cost where c_i is the cost of i -th step on a staircase. Once you pay the cost, you can either climb one or two steps.
- You can either start from the step with index 0, or the step with index 1.
- Return the minimum cost to reach the top of the floor.

Recurrence relation

$$sol(n) = \begin{cases} 0, & \text{if } n=0 \\ 0, & \text{if } n=1 \\ \min(sol(n-1) + c_{n-1}, sol(n-2) + c_{n-2}), & n > 2 \end{cases} \quad (2)$$

[Click here for code and Leetcode problem description](#)

Turn the idea into actual code

```
1 public int minCostClimbingStairs(int[] cost) {  
2     int n = cost.length;  
3     Integer[] memo = new Integer[n + 1];  
4     return dp(n, cost, memo);  
5 }  
  
7 private int dp(int i, int[] cost, Integer[] memo) {  
8     if (i == 0 || i == 1) return 0;  
9  
10    if (memo[i] != null) return memo[i];  
11  
12    int oneStep = dp(i - 1, cost, memo) + cost[i - 1];  
13    int twoStep = dp(i - 2, cost, memo) + cost[i - 2];  
14  
15    return memo[i] = Math.min(oneStep, twoStep);  
16 }
```

- Time complexity is $O(n)$ because memorization is used

Outline

- 1 Fibonacci Number: Motivating Example for DP
 - Variations of Fibonacci Number: Climb stairs
 - Distinct ways to climb stairs
 - Minimum cost stair climbing
- 2 Extension of stair climbing: Frog jump
 - Frog Jump with missing stone
 - Frog Jump with energy supply
 - Number of sums
- 3 2D paths
- 4 Knapsack problem
 - Recursive solution
 - Tabulated solution



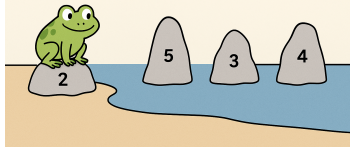
Frog jump (min energy)

question at [DMOJ](#) and [keeksforgeeks](#)

Problem

Given n stones in a line each having height $h[i], i \in \{0, 1, \dots, n-1\}$. A frog wants to jump from the $0th$ stone to the $(n-1)th$ stone with minimum energy consumption. Whenever the frog jumps from a stone i to stone j , the energy consumed in the jump is $abs(h[i] - h[j])$. At a time the frog can jump either one or two stones.

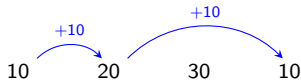
Frogo the frog wants to cross the Blue River.
Each stone has a different height, and every
jump costs energy.



Example

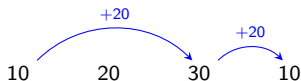
Given four stones of heights (10, 20, 30, 10).

- One jump sequence is



The energy cost is $10 + 10 = 20$.

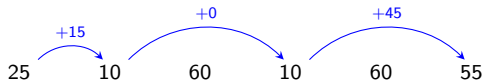
- Another sequence is



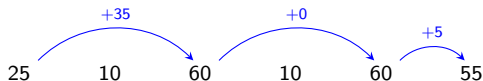
The energy cost is $20 + 20 = 40$.

Greedy approach does not work

- Greedy algorithm?
 - each time choose the stone with minimum energy per step
- A counter example:

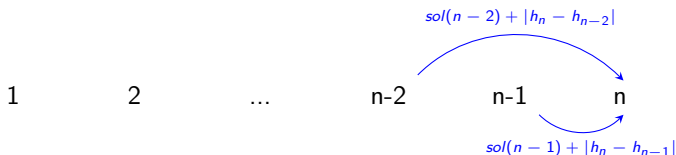


- The result is worse than the jumps below:



- Hence, we need to come up with a better algorithm than greedy choices.

Recurrence relation



There are two ways to reach stone n — jump from stone $n-1$ or from stone $n-2$. Suppose that the optimal solution for $n-1$ and $n-2$ are solved by $dp(n-1)$ and $dp(n-2)$ respectively.

$$sol(n) = \begin{cases} 0, & \text{if } n=0 \\ h_0, & \text{if } n=1 \\ \min(sol(n-1) + |h_n - h_{n-1}|, sol(n-2) + |h_n - h_{n-2}|), & n > 1 \end{cases}$$

- Similar to the fib example, we can translate the recurrence relation into code.
- The complexity of the program is $O(2^n)$.
- Need to use memorization to skip repeated computations

Frog jump

Variation of the problem

- k-step jump ([DMOJ](#))
- different cost ([DMOJ](#))
- ..

Outline

- 1 Fibonacci Number: Motivating Example for DP
 - Variations of Fibonacci Number: Climb stairs
 - Distinct ways to climb stairs
 - Minimum cost stair climbing
- 2 Extension of stair climbing: Frog jump
 - Frog Jump with missing stone
 - Frog Jump with energy supply
 - Number of sums
- 3 2D paths
- 4 Knapsack problem
 - Recursive solution
 - Tabulated solution

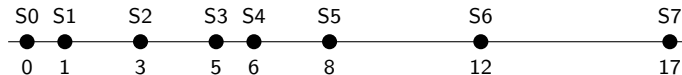
Frog jump with missing stones (LeetCode)

A frog is crossing a river. The river is divided into x units and at each unit there may or may not exist a stone. The frog can jump on a stone, but it must not jump into the water.

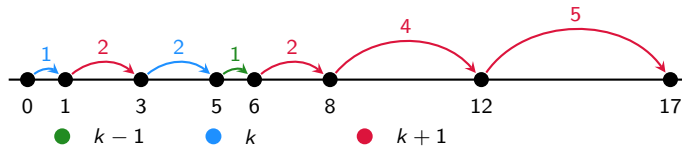
Given a list of stones' positions (in units) in sorted ascending order, determine if the frog is able to cross the river by landing on the last stone. Initially, the frog is on the first stone and assume the first jump must be 1 unit. If the frog has just jumped k units, then its next jump must be either $k - 1$, k , or $k + 1$ units. Frog can only jump in the forward direction.

- Example 1: Given 8 stones $[0, 1, 3, 5, 6, 8, 12, 17]$. The answer is true with sequence $(0, 1, 3, 5, 8, 12)$,
- Example 2: Given 8 stones $[0, 1, 2, 3, 4, 8, 9, 11]$. The answer is false because the gap between 4 and 8 is too big.

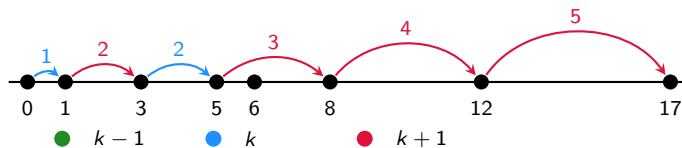
Stone positions

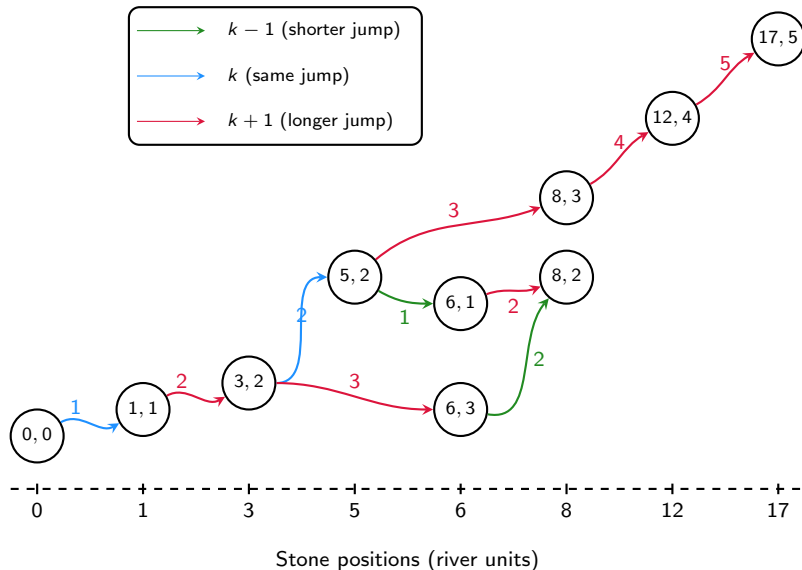


Invalid path:



Valid path:





Recurrence relation

$$dp(i, k) = \begin{cases} \text{true,} & \text{if the frog can land on stone } i \text{ with last jump length } k, \\ \text{false,} & \text{otherwise.} \end{cases}$$

The frog starts on stone 0 with a last jump of length 0:

$$dp(0, 0) = \text{true.}$$

The goal is to determine whether there exists any k such that

$$dp(n - 1, k) = \text{true,}$$

where $n - 1$ is the index of the last stone.

$$dp(j, k) = dp(i, k - 1) \vee dp(i, k) \vee dp(i, k + 1),$$

Corresponding code

```
2
3
4 class Solution {
5     public boolean canCross(int[] stones) {
6         return dp(0, 0, stones);
7     }
8
9     private boolean dp(int i, int k, int[] s) {
10         if (i == s.length - 1) return true;
11
12         for (int step = k - 1; step <= k + 1; step++) {
13             int j = Arrays.binarySearch(s, s[i]+step); \\find the index
14             if (j >= 0 && dp(j, step, s))
15                 return true;
16         }
17         return false;
18     }
19 }
```

- It is $O(2^n)$
- Add memorization will make it $O(n^2)$.

With memorization

- Lecture note on this problem [\[PDF\]](#)

```
class Solution {
    public boolean canCross(int[] stones) {
        int n = stones.length;
        Boolean[][] memo = new Boolean[n][n + 1];
        return dfs(0, 0, stones, memo);
    }

    // dp(i, k): can we stand on stone i with last jump length k?
    private boolean dfs(int i, int k, int[] s, Boolean[][] memo) {
        if (i == s.length - 1) return true;
        if (memo[i][k] != null) return memo[i][k];

        for (int step = k - 1; step <= k + 1; step++) {
            if (step <= 0) continue;

            int nextPos = s[i] + step;
            int j = java.util.Arrays.binarySearch(s, nextPos);

            if (j >= 0 && dfs(j, step, s, memo)) {
                return memo[i][k] = true;
            }
        }

        return memo[i][k] = false;
    }
}
```

Outline

- 1 Fibonacci Number: Motivating Example for DP
 - Variations of Fibonacci Number: Climb stairs
 - Distinct ways to climb stairs
 - Minimum cost stair climbing
- 2 Extension of stair climbing: Frog jump
 - Frog Jump with missing stone
 - Frog Jump with energy supply
 - Number of sums
- 3 2D paths
- 4 Knapsack problem
 - Recursive solution
 - Tabulated solution

Frog jump with energy supply

Problem

Same as before but on some stones, there may be worms, which add energy to the frog. If the frog jumps on a stone with a worm on it, it **MUST** eat the worm. The list $T[0...n-1]$ contains the energy values of the worms on the corresponding stones. If there is no worm on the stone, this is equivalent to a value of 0.

The frog starts with 0 Energy, but it is guaranteed that there is a worm on stone number 0.

- Given a list T , the task is to return a list of the indexes of the stones on which the frog has been, before getting to stone $n - 1$.
- However, this must be the shortest possible list. In some cases there may be more than one solution, whichever is accepted.
- It is guaranteed that always the frog will be able to get to stone $n - 1$.
- Example no. 1: For $T = [3, 0, 2, 1, 0, 2, 5, 0]$ the answer is $[0, 2, 5]$.
- Example no. 2: For $T = [7, 0, 0, 1, 0, 3, 0, 6, 0, 0, 0, 0, 0, 0, 0]$ the answer is $[0, 3, 7]$ or $[0, 5, 7]$.

Outline

1 Fibonacci Number: Motivating Example for DP

- Variations of Fibonacci Number: Climb stairs
 - Distinct ways to climb stairs
 - Minimum cost stair climbing

2 Extension of stair climbing: Frog jump

- Frog Jump with missing stone
- Frog Jump with energy supply
- **Number of sums**

3 2D paths

4 Knapsack problem

- Recursive solution
- Tabulated solution

Number of sums

Problem

Given a number N . Find the number of different ways to write it as the sum of 1, 3 and 4.

Example

if $N = 5$, the answer would be 6, because

$$1 + 1 + 1 + 1 + 1 \quad (1)$$

$$1 + 4 \quad (2)$$

$$4 + 1 \quad (3)$$

$$1 + 1 + 3 \quad (4)$$

$$1 + 3 + 1 \quad (5)$$

$$3 + 1 + 1 \quad (6)$$

Number of sums

Consider one possible solution,

$$n = x_1 + x_2 + \dots x_n. \quad (7)$$

- If $x_n = 1$, the sum of the remaining numbers should be $n - 1$. So, number of sums that end with 1 is equal to $DP(n-1)$.
- If $x_n = 3$, we need to find the solution for $DP(n-3)$.
- if $x_n = 4$, there are $DP(n-4)$ solutions.

Sum those cases, we have

$$DP(n) = DP(n-1) + DP(n-3) + DP(n-4). \quad (8)$$

The corresponding code is:

```
DP[0] = DP[1] = DP[2] = 1;
DP[3] = 2;
for (i = 4; i <= n; i++) {
    DP[i] = DP[i-1] + DP[i-3] + DP[i-4];
}
```

time complexity?

The recursive version

```
1  int[] memo;
3  int solve(int n) {
4      memo = new int[n + 1];
5      Arrays.fill(memo, -1);
6      return dp(n);
7  }
9  int dp(int i) {
10     if (i <= 2) return 1;
11     if (i == 3) return 2;
13     if (memo[i] != -1) return memo[i];
15     // recurrence: DP[i] = DP[i-1] + DP[i-3] + DP[i-4]
16     memo[i] = dp(i - 1) + dp(i - 3) + dp(i - 4);
17     return memo[i];
18 }
```

Outline

- 1 Fibonacci Number: Motivating Example for DP
 - Variations of Fibonacci Number: Climb stairs
 - Distinct ways to climb stairs
 - Minimum cost stair climbing
- 2 Extension of stair climbing: Frog jump
 - Frog Jump with missing stone
 - Frog Jump with energy supply
 - Number of sums
- 3 2D paths
- 4 Knapsack problem
 - Recursive solution
 - Tabulated solution

Finding Minimum-Cost Path in a 2D Matrix

Problem

Given a cost matrix $Cost[][]$ where $Cost[i][j]$ denotes the Cost of visiting cell with coordinates (i,j) , find a min-cost path to reach a cell (x,y) from cell $(0,0)$ under the condition that you can only travel one step right or one step down. (costs are positive integers)

Solution

If you reach a position (i,j) in the grid, you must have come from one cell higher, i.e. $(i-1,j)$ or from one cell to your left, i.e. $(i,j-1)$. This means that the cost of visiting cell (i,j) will come from the following recurrence relation:

$$MinCost(i,j) = \min(MinCost(i-1,j), MinCost(i,j-1)) + Cost[i][j] \quad (9)$$

Code without memorization

```
1 int minCost(int [][] cost, int x, int y) {  
2     if (x < 0 || y < 0) return Integer.MAX_VALUE / 2;  
3     if (x == 0 && y == 0) return cost[0][0];  
4  
5     int fromTop = minCost(cost, x - 1, y);  
6     int fromLeft = minCost(cost, x, y - 1);  
7  
8     return cost[x][y] + Math.min(fromTop, fromLeft);  
9 }
```

DP version

```
1  int [][] cost, memo;
3  int minCost(int [][] c, int x, int y) {
4      cost = c;
5      memo = new int[c.length][c[0].length];
6      for (int[] row : memo) Arrays.fill(row, -1);
7      return dfs(x, y);
8  }
9
10 int dfs(int i, int j) {
11     if (i < 0 || j < 0) return Integer.MAX_VALUE / 2;
12     if (i == 0 && j == 0) return cost[0][0];
13     if (memo[i][j] != -1) return memo[i][j];
14
15     return memo[i][j] = cost[i][j] + Math.min(dfs(i - 1, j), dfs(i, j - 1));
16 }
```

Outline

- 1 Fibonacci Number: Motivating Example for DP
 - Variations of Fibonacci Number: Climb stairs
 - Distinct ways to climb stairs
 - Minimum cost stair climbing
- 2 Extension of stair climbing: Frog jump
 - Frog Jump with missing stone
 - Frog Jump with energy supply
 - Number of sums
- 3 2D paths
- 4 Knapsack problem
 - Recursive solution
 - Tabulated solution

0/1 Knapsack problem



Problem

Given n items, with item i being worth v_i and having weight w_i . Fill a knapsack of capacity W with maximal value. What is the maximal value?

- It's called 0/1 knapsack problem because the items can not be divided or repeated.
- For simplicity purpose, we return the max value only. A variant is to return the selected item list.
- Greedy strategy does not work: picking the item with the biggest value-per-pound might work? Here is a counter-example showing that this does not work: ...

Divide the problem into a subproblem

Let's look at the last item. It can be either selected or not. When it is selected, the total capacity is reduced by $w[n]$. Otherwise, the capacity is not changed. Hence the total value of the knapsack problem for n items with capacity c has two cases:

$$T(n, c) = \begin{cases} 0, & \text{if } n=0 \\ T(n-1, c), & \text{if } n\text{-th item not taken} \\ v_n + T(n-1, c - w_n), & \text{if } n\text{-th item is taken} \end{cases}$$

What we want is to choose the maximum

$$T(n, c) = \max(T(n-1, c), v_n + T(n-1, c - w_n))$$

Hence we have the program:

```
int knapsack(int[] w, int[] v, int c, int n) {  
    if (n <= 0 || c <= 0) return 0;  
    if (w[n-1] > c) return knapsack(w, v, c, n-1);  
    return Math.max(knapsack(w, v, c, n-1), v[n-1] + knapsack(w, v, c-w[n-1], n-1));  
}
```

The time complexity of the algorithm?

Recursion with memorization

```
1  Integer [][] dp = new Integer[v.length][capacity + 1];  
3  
5  int knapsack(Integer [][] dp, int[] v, int[] w, int c, int n) {  
6      if (c <= 0 || n >= v.length) return 0;  
7      if(dp[n][c] != null) return dp[n][c];  
8      if( w[n] <= c )  
9          int profit1 = v[n] + knapsack(dp, v, w, c-w[n], n-1);  
10         int profit2 = knapsack(dp, v, w, c, n-1);  
11         dp[n][c] = Math.max(profit1, profit2);  
12     return dp[n][c];  
13 }
```

[Here](#) you can find complete code in various languages.

Bottom up tracing

$$T(n, c) = \begin{cases} T(n-1, c), & \text{n-th item not taken} \\ v_n + T(n-1, c - w_n), & \text{n-th item is taken} \end{cases}$$

$$T(n, c) = \max(T(n-1, c), v_n + T(n-1, c - w_n))$$

weight	value	item	Capacity									
			0	1	2	3	4	5	6	7	8	
			0	0	0	0	0	0	0	0	0	
2	1	1	0									
3	2	2	0									
4	5	3	0									
5	6	4	0									

For item 1, what is the total value under each column (capacity)? Since $w_1 = 2$, we can take item 1 only when $c \geq 2$.

(10)

Bottom up tracing

$$T(n, c) = \begin{cases} T(n-1, c), & \text{n-th item not taken} \\ v_n + T(n-1, c - w_n), & \text{n-th item is taken} \end{cases}$$

$$T(n, c) = \max(T(n-1, c), v_n + T(n-1, c - w_n))$$

weight	value	item	Capacity								
			0	1	2	3	4	5	6	7	8
			0	0	0	0	0	0	0	0	0
2	1	1	0	0	1	1	1	1	1	1	1
3	2	2	0								
4	5	3	0								
5	6	4	0								

$$T(1, 1) = \max(T(1-1, 1), v_1 + T(1-1, 1 - w_1)) = \max(T(0, 1), v_1 + T(0, -1)) = 0$$

$$T(1, 2) = \max(T(1-1, 2), v_1 + T(1-1, 2 - w_1)) = \max(T(0, 2), v_1 + T(0, 0)) = 1$$

...

$$T(1, 8) = \max(T(1-1, 8), v_1 + T(1-1, 8 - w_1)) = \max(T(0, 8), v_1 + T(0, 6)) = 1$$

Bottom up tracing

$$T(n, c) = \begin{cases} T(n-1, c), & \text{n-th item not taken} \\ v_n + T(n-1, c - w_n), & \text{n-th item is taken} \end{cases}$$

$$T(n, c) = \max(T(n-1, c), v_n + T(n-1, c - w_n))$$

weight	value	item	Capacity								
			0	1	2	3	4	5	6	7	8
			0	0	0	0	0	0	0	0	0
2	1	1	0	0	1	1	1	1	1	1	1
3	2	2	0	0	1	2	2	3	3	3	3
4	5	3	0								
5	6	4	0								

$$T(2, 1) = \max(T(1, 1), v_2 + T(1, 1 - w_2)) = \max(T(1, 1), v_2 + T(1, -2)) = 0$$

$$T(2, 2) = \max(T(1, 2), v_2 + T(1, 2 - w_2)) = \max(T(1, 2), v_2 + T(1, -1)) = \max(1, 0) = 1$$

$$T(2, 3) = \max(T(1, 3), v_2 + T(1, 3 - w_2)) = \max(T(1, 3), v_2 + T(1, 0)) = \max(1, 2)$$

...

$$T(2, 8) = \max(T(1, 8), v_2 + T(1, 8 - w_2)) = \max(T(1, 8), v_2 + T(1, 5)) = \max(1, 2 + 1) = 3$$

Bottom up tracing

$$T(n, c) = \begin{cases} T(n-1, c), & \text{n-th item not taken} \\ v_n + T(n-1, c - w_n), & \text{n-th item is taken} \end{cases}$$

$$T(n, c) = \max(T(n-1, c), v_n + T(n-1, c - w_n))$$

weight	value	item	Capacity								
			0	1	2	3	4	5	6	7	8
			0	0	0	0	0	0	0	0	0
2	1	1	0	0	1	1	1	1	1	1	1
3	2	2	0	0	1	2	2	3	3	3	3
4	5	3	0	0	1	2	5	5	6	7	7
5	6	4	0								

$$T(3, 1) = \max(T(2, 1), v_3 + T(2, 1 - w_3)) = \max(0, 5 + T(2, -3)) = 0$$

$$T(3, 2) = \max(T(2, 2), v_3 + T(2, 2 - w_3)) = \max(1, 5 + T(2, -2)) = \max(1, 0) = 1$$

$$T(3, 3) = \max(T(2, 3), v_3 + T(2, 3 - w_3)) = \max(2, 5 + T(2, -1)) = \max(1, 2)$$

$$T(3, 4) = \max(T(2, 4), v_3 + T(2, 4 - w_3)) = \max(2, 5 + T(2, 0)) = \max(1, 5 + 0)$$

$$T(3, 5) = \max(T(2, 5), v_3 + T(2, 5 - w_3)) = \max(3, 5 + T(2, 1)) = \max(1, 5 + 0)$$

$$T(3, 6) = \max(T(2, 6), v_3 + T(2, 6 - w_3)) = \max(3, 5 + T(2, 2)) = \max(1, 5 + 1)$$

$$T(3, 7) = \max(T(2, 7), v_3 + T(2, 7 - w_3)) = \max(3, 5 + T(2, 3)) = \max(1, 5 + 2)$$

$$T(3, 8) = \max(T(2, 8), v_3 + T(2, 8 - w_3)) = \max(3, 5 + T(2, 4)) = \max(1, 5 + 2) = 7$$

Bottom up tracing

$$T(n, c) = \begin{cases} T(n-1, c), & \text{n-th item not taken} \\ v_n + T(n-1, c - w_n), & \text{n-th item is taken} \end{cases}$$

$$T(n, c) = \max(T(n-1, c), v_n + T(n-1, c - w_n))$$

weight	value	item	Capacity								
			0	1	2	3	4	5	6	7	8
			0	0	0	0	0	0	0	0	0
2	1	1	0	0	1	1	1	1	1	1	1
3	2	2	0	0	1	2	2	3	3	3	3
4	5	3	0	0	1	2	5	5	6	7	7
5	6	4	0	0	1	2	5	6	6	7	8

$$T(4, 1) = \max(T(3, 1), v_4 + T(3, 1 - w_4)) = \max(0, 6 + T(3, -4)) = 0$$

$$T(4, 2) = \max(T(3, 2), v_4 + T(3, 2 - w_4)) = \max(1, 6 + T(3, -3)) = \max(1, 0) = 1$$

$$T(4, 3) = \max(T(3, 3), v_4 + T(3, 3 - w_4)) = \max(2, 6 + T(3, -2)) = \max(2, 0)$$

$$T(4, 4) = \max(T(3, 4), v_4 + T(3, 4 - w_4)) = \max(2, 6 + T(3, -1)) = \max(2, 0)$$

$$T(4, 5) = \max(T(3, 5), v_4 + T(3, 5 - w_4)) = \max(2, 6 + T(3, 0)) = \max(2, 6) = 6$$

$$T(4, 6) = \max(T(3, 6), v_4 + T(3, 6 - w_4)) = \max(2, 6 + T(3, 1)) = \max(2, 6) = 6$$

$$T(4, 7) = \max(T(3, 7), v_4 + T(3, 7 - w_4)) = \max(2, 6 + T(3, 2)) = \max(2, 6 + 1) = 7$$

$$T(4, 8) = \max(T(3, 8), v_4 + T(3, 8 - w_4)) = \max(3, 6 + T(3, 3)) = \max(1, 6 + 2) = 8$$

Iterative approach

Based on the trace, we can develop the following iterative program.

```
1 int knapsackDP(int[] w, int[] v, int n, int c) {  
2     if (n <= 0 || c <= 0) return 0;  
3     int[][] m = new int[n + 1][c + 1];  
4     for (int j = 0; j <= c; j++) m[0][j] = 0;  
5  
6     for (int i = 1; i <= n; i++) {  
7         for (int j = 1; j <= c; j++) {  
8             if (w[i - 1] > j) {  
9                 m[i][j] = m[i - 1][j];  
10            } else {  
11                m[i][j] = Math.max(  
12                    m[i - 1][j],  
13                    m[i - 1][j - w[i - 1]] + v[i - 1]);  
14            }  
15        }  
16    }  
17    return m[n][c];  
18 }
```